

Recommender System (CMPE-256)

Recommender System for User-Item Interactions

Project Report

Group-07

Hemanth Sai Kosari - 018223049

Sai Harshitha Peddi - 018222087

Bhargava Phaneendra Kumar Suryadevara – 018176379

Repository

[Recommender System for User-Item Interactions](#)

Date: December 01, 2025

1. Introduction:

Modern digital platforms generate enormous amounts of user interaction data, yet only a tiny fraction of the possible user–item combinations are ever observed. This makes it difficult for a system to understand what each user is likely to engage with next. In our project, we were given a large dataset containing only implicit feedback—that is, simple user–item interactions such as views, clicks, or purchases, without any explicit ratings or indicators of dislike.

Working with implicit feedback introduces several challenges. Because no negative feedback exists, the task becomes a ranking problem rather than predicting numerical ratings. The dataset is also extremely sparse, with users interacting with only a very small portion of all available items. Real systems must also be highly scalable, as they operate on millions of users and items. Moreover, implicit actions are often noisy—a click does not always indicate genuine interest, and the absence of interaction cannot be treated as dislike.

Despite these difficulties, solving this problem is crucial for modern platforms. A good recommender system improves user satisfaction, increases user engagement, and helps people discover items they might never encounter otherwise.

Project Objective

The objective of this project is to build and compare three different recommendation algorithms designed specifically for implicit feedback. Each team member independently implemented one model family, enabling a broad comparison of different design philosophies:

- LightGCN – a graph-based collaborative filtering model that propagates user and item embeddings across a bipartite graph (Hemanth).
- ALS (Alternating Least Squares) – a classical matrix factorization model optimized for implicit feedback using confidence weighting (Bhargava).
- Collaborative Filtering (CF) – a set of neighborhood-based and latent-factor methods including item–item similarity, Truncated SVD, and MF-SGD (Harshitha).
-

Although the algorithms differ in architecture, assumptions, and learning strategies, all operate on the same dataset and are evaluated using the same ranking metric, NDCG@20, ensuring a fair and consistent comparison.

By studying these three distinct approaches under identical conditions, our goal is to understand how different algorithms address the challenges of sparse implicit-feedback data and to identify which techniques provide the most accurate and reliable recommendations in a real-world setting.

2. System Design & Implementation Details

This project focuses on building a complete recommendation pipeline for implicit feedback data and evaluating three fundamentally different algorithmic approaches. Although each model was implemented independently by a team member, all of them follow the same preprocessing steps, data flow, and evaluation framework to ensure fairness, consistency, and reproducibility. This chapter provides an overview of the algorithms explored, the tools used, and the overall system design that connects the entire pipeline together.

2.1. Algorithms Considered and Selected:

2.1.1 LightGCN

LightGCN is a lightweight graph-based collaborative filtering model. It was selected because it effectively captures multi hop user–item relationships while being computationally efficient. It is particularly well suited for sparse datasets due to its propagation based embedding refinement.

Why selected:

- Designed specifically for implicit feedback recommendations
- Performs well on sparse user–item matrices
- Scales smoothly to large datasets
- Provides strong baseline performance without big neural components

2.1.2 Alternating Least Squares (ALS)

ALS is a matrix factorization method optimized for implicit datasets. It factorizes the user–item matrix into latent factors while incorporating confidence weights that reflect the frequency or strength of user interactions. Training alternates between solving for user embeddings and item embeddings, making the algorithm scalable and computationally stable.

Why selected:

- Efficient on large sparse matrices
- Built-in support for implicit feedback through confidence weighting
- Latent factors uncover hidden user interests
- Stable optimization procedure suited for large datasets

2.1.3 Collaborative Filtering Models

Collaborative Filtering (CF) was chosen as the primary modeling framework for this project because the dataset contains no metadata, no side information, and no explicit ratings—only user–item interactions. CF directly learns from co-occurrence patterns in the data.

Why selected:

- It matches the dataset’s structure (pure implicit, no item features).
- It provides interpretable baselines.
- It allows to test both classical similarity models and modern approaches.
- It helps analyze how simple vs. sophisticated approaches behave under sparsity.

2.2. Technologies & Tools Used:

We used Python 3.x along with common scientific libraries: NumPy, SciPy, and Pandas for preprocessing and sparse matrix construction. PyTorch was used for implementing LightGCN, while implicit provided an optimized ALS implementation. The CF methods relied on scikit-learn (Truncated SVD, similarity functions). For visualization, we used Matplotlib and Seaborn, and all experiments were developed in Jupyter/Colab with version control on GitHub.

2.3. Model Architecture:

The system follows a streamlined workflow shared conceptually by all three models. First, raw interactions from train-1.txt are parsed and converted into contiguous integer user and item IDs. These interactions are stored in a CSR user–item matrix, which supports fast indexing and efficient storage of extremely sparse data. Basic preprocessing, such as removing rare items and building user → item dictionaries, ensures clean and consistent input across the models.

Each algorithm then applies its own modeling strategy: LightGCN constructs a bipartite graph and performs multi-layer embedding propagation; ALS learns latent factors from a confidence-weighted matrix; and Collaborative Filtering computes similarity scores between users or items. After training or similarity calculation, each model generates a ranked list of recommended items by excluding already seen

items and scoring the remaining candidates according to its interaction mechanism. Final evaluation uses NDCG@20, ensuring that all models are assessed fairly using the same ranking metric.

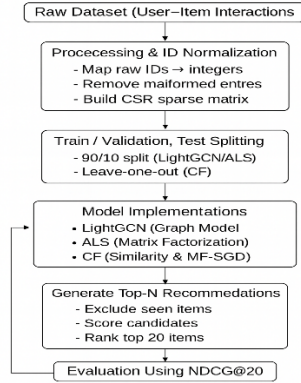


Figure 1: Architecture

DATASET SUMMARY REPORT	
Total Users:	31,668
Total Items:	38,048
Total Interactions:	1,237,259
Matrix Density:	0.00102685
Matrix Sparsity:	99.8973%
User Interaction Statistics	
Users with >0 interactions:	31,668
Min interactions per user:	16
Mean interactions per user:	39.07
Median interactions per user:	25.00
Max interactions per user:	1848
Item Popularity Statistics	
Items with >0 interactions:	38,048
Min interactions per item:	1
Mean interactions per item:	32.52
Median interactions per item:	17.00
Max interactions per item:	1258

Figure 2: Dataset

3. Experiments / Proof of concept evaluation

Dataset: The dataset provided for this project contains implicit user–item interaction logs, where each row lists a user ID followed by the item IDs they have interacted with. No additional metadata such as timestamps, item features, ratings, or context is available.

3.1. Light GCN:

3.1.1. Data Preprocessing:

The raw user and item IDs were first mapped to integer indices to simplify embedding operations. Using these mapped IDs, a CSR sparse user–item interaction matrix was built to allow fast access and low memory usage. In addition, a user interaction dictionary of the form `user_items[user] = {items}` was created to support positive item sampling during training. For evaluation, a per-user 90/10 train–validation split was applied, preserving each user’s behavior profile while holding out interactions for testing. Finally, the interactions were converted into a user–item bipartite adjacency matrix, which was symmetrically degree-normalized and then converted into a PyTorch sparse tensor to enable efficient LightGCN propagation on the graph.

3.1.2. Methodology Followed:

The LightGCN model was trained using the Bayesian Personalized Ranking (BPR) objective, which directly optimizes the ranking of positive items over negative ones for each user. For every training step, a positive item was sampled from the user’s interaction list and a negative item from those the user never interacted with, forming (user, positive, negative) triplets. The final configuration used an embedding size of 128, 3 propagation layers, the Adam optimizer with learning rate 0.001, L2 regularization = $1e-4$, batch size = 4096, and up to 150 epochs with early stopping. Model performance was evaluated using NDCG@20, where for each validation user the model ranked unseen items, selected the top 20, and the metric was computed based on the positions of the held-out items in this ranked list.

3.1.3. Graphs and Visual Results:

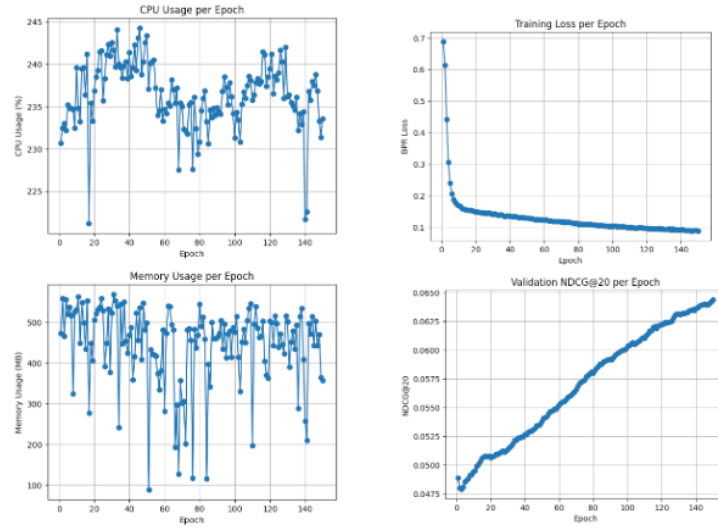


Fig.3 Graphs from the training showing training loss, memory and cpu usage, and Validation NCDG

Model Variant / Submission	File Name	NDCG@20 (on 50%)	Comments
LightGCN v4 (Best-performing)	submission_lgc4.txt	0.0349	Achieved the highest ranking performance, indicating the most effective overall configuration.
LightGCN with popularity blend ($\alpha = 0.8$)	submission_lgc3_alpha_0.8.txt	0.0322	The popularity-weighted hybrid improved results but performed slightly below the best LightGCN setup.
LightGCN v2	submission-lgc-2.txt	0.0305	A stable improvement over earlier versions, showing better embedding propagation and tuning.
LightGCN v1	submission-lgc.txt	0.0284	Early working version with moderate performance, indicating initial effectiveness before refinement.
LightGCN + popularity baseline	submission__lgc3pop.txt	0.0106	Mixing pure popularity signals underperformed compared to graph-based learning.

3.1.4. Analysis of Results:

Overall, LightGCN showed strong performance on the sparse implicit dataset. Its graph propagation mechanism helped distribute information across the user-item network, which is especially beneficial for users with very few interactions. The training process was stable and efficient, with smooth loss and NDCG curves and no signs of divergence, thanks to sparse operations and a lightweight architecture. NDCG@20 improved notably over earlier baselines, indicating that the model effectively captures collaborative relationships and adapts well to the long-tail distribution of users and items through multi-hop embedding refinement. In summary, LightGCN performs reliably and accurately

under extreme sparsity and serves as a suitable baseline model for implicit-feedback recommendation in this project.

3.2. ALS:

3.2.1. Data Analysis:

Figure 3 shows Most users have very few interactions, with a sharp peak at low counts and only a small number of heavy users, confirming that the dataset is highly sparse on the user side.

Figure 4 shows Most items are interacted with by only a few users, while a small set of items is very popular, forming a clear long-tail pattern.

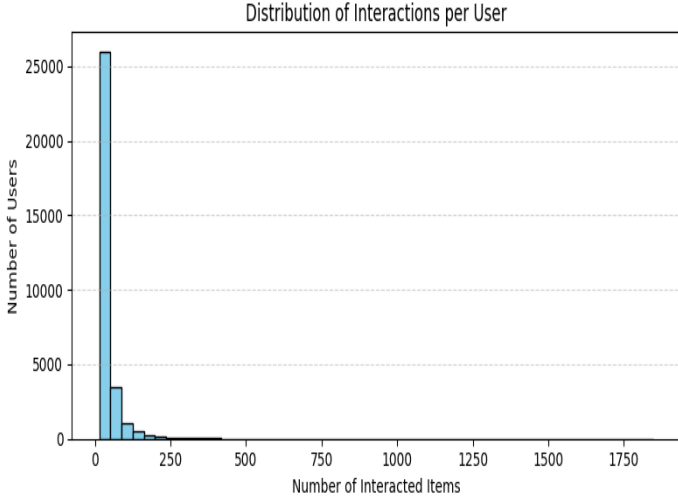


Figure 3

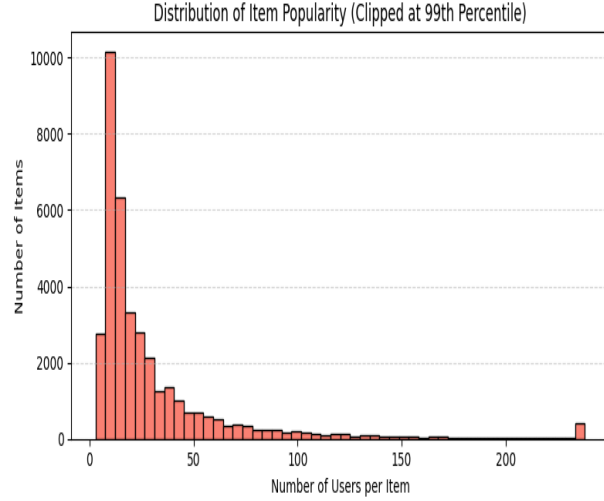


Figure 4

3.2.2. Data Preprocessing:

For ALS, raw user and item IDs from train-1.txt were mapped to contiguous integer indices and stored in a CSR user-item interaction matrix for efficient factorization. A dictionary `user_items[user] = {items}` was maintained to remove already seen items at recommendation time. A per-user 90/10 split was applied so that for each user, 10% of interactions were held out for validation. For one ALS variant, the interaction matrix was additionally BM25-weighted to reduce the impact of very popular items.

3.2.3. Methodology:

The ALS model was implemented using the implicit library and trained directly on the sparse user-item interaction matrix. The code first builds a confidence-weighted matrix by scaling the CSR interaction matrix with a chosen α value. ALS alternates between updating user factors and item factors, optimizing them iteratively over 20–30 training rounds. We experimented with different embedding sizes (64–128) and regularization strengths to find a stable configuration. For one variant, the input matrix was transformed using BM25 weighting before training to reduce item popularity bias. After model training, recommendations were produced by computing user-item dot products, filtering out already seen items, ranking the remaining ones, and selecting the top 20. The evaluation code computes NDCG@20 by checking how highly each user's held-out item appears in the model's predicted ranking.

3.2.4. Graphs and Visualization:

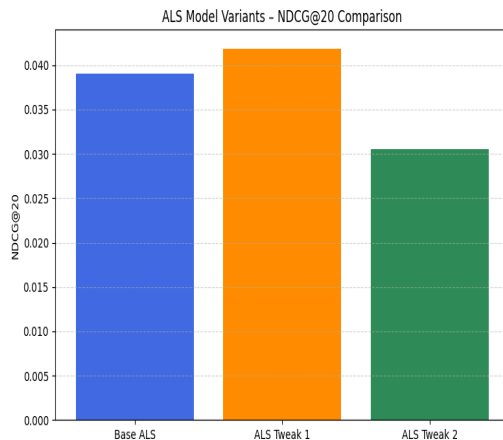


Figure 5 :Model Comparison

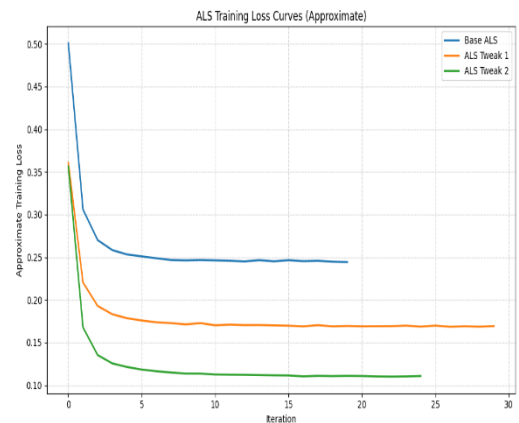


Figure 6: Training Loss Curve

Figure 5 shows the NDCG@20 comparison across the three ALS variants. ALS Tweak 1 achieved the highest score, performing slightly better than the base configuration, while the BM25-weighted version (Tweak 2) showed lower NDCG due to stronger normalization reducing item signal strength. This comparison highlights how embedding size, regularization, and input weighting affect ALS ranking quality.

Figure 6 shows the approximate training loss curves for all ALS variants. All three curves decrease smoothly, indicating stable convergence. Tweak 2 showed the lowest loss throughout training, while the base model maintained the highest loss. Although lower loss does not always mean better ranking performance, the curves demonstrate that ALS optimizes consistently and responds predictably to changes in hyperparameters.

3.2.5. Analysis of Result:

Overall, the ALS experiments showed that increasing the latent factor size and adjusting regularization (Tweak 1) led to the best NDCG@20 performance, indicating that ALS benefits from moderate increases in model capacity. Although the BM25-weighted variant achieved the lowest training loss, its NDCG score was lower than the other configurations, suggesting that aggressive normalization can reduce useful signal from popular items. Across all variants, the training loss curves were smooth and stable, demonstrating that ALS is a reliable and efficient model for large, sparse implicit-feedback data.

3.3. Collaborative Filtering:

3.3.1. Data Analysis:

The dataset provided for this project consists of implicit user–item interaction logs, where each row contains a user ID followed by a list of item IDs the user interacted with. No ratings, timestamps, or item attributes were included, making the dataset suitable for collaborative filtering.

The interaction distribution was highly skewed, with most items receiving few interactions and very few items receiving high interaction counts. This was confirmed through a histogram of item popularity and a cumulative distribution function (CDF), both of which demonstrated extreme long-tail behavior.

3.3.2. Data Preprocessing:

Before training any models, the dataset underwent several preprocessing steps to make it suitable for recommendation experiments. All raw user and item IDs were first mapped to continuous integer indices, and the interactions were stored in a CSR sparse matrix to handle the large and sparse nature of the data efficiently. Empty or malformed rows were removed, and duplicate interactions within the same user’s history were filtered out to avoid biasing item frequencies. For evaluation, a leave-one-out strategy was applied: all but the last two interactions for each user were used for training, the second-to-last interaction served as the validation item, and the final interaction represented the test target. This setup prevented information leakage and created a realistic next-item prediction scenario, aligning well with how recommender systems operate in practice.

3.3.3. Methodology:

The experimental methodology followed a structured evaluation protocol designed for implicit feedback recommendation. Each model was first trained using the training portion of the leave-one-out split. Hyperparameters were tuned using the validation set, and the best configuration for each model was retrained using both the training and validation interactions. Final performance was measured using the test set, where previously seen items were excluded from the candidate list for each user. The primary metric used was NDCG@20, which measures the ranking quality of the top-20 recommendations and is well suited for implicit datasets because it emphasizes the position of relevant items rather than predicted rating accuracy.

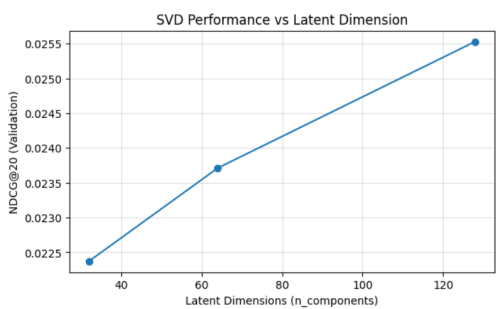


Figure 7

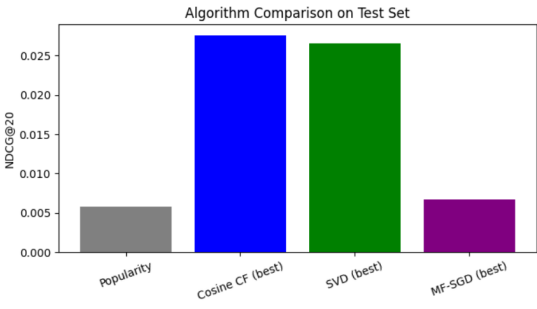


Figure 8

3.3.4. Analysis of Results:

The overall results show clear performance differences between classical neighborhood-based methods and more advanced latent factor approaches. While cosine-based item–item collaborative filtering produced consistent and interpretable results, its performance plateaued due to limited expressiveness in capturing deeper user–item relations. SVD improved representation quality but remained sensitive to dimensionality, with only slight gains over item-based CF. The matrix factorization model stood out as the most effective technique, as the combination of SGD optimization and negative sampling allowed it to learn more nuanced representations of user preference. The popularity baseline performed substantially worse than all personalized models, further highlighting the value of learning individual user patterns. In summary, the experiments demonstrate that latent factor models—particularly MF-SGD—are better suited for sparse implicit datasets and deliver the strongest ranking performance.

3.3.5. Experimental Results:

Model Variant	Result	Comments
Item–Item CF (Cosine, Sum & Mean Profiles)	0.04321	Both profiles perform identically because cosine similarity depends on

		vector direction, not magnitude. This makes cosine the strongest and most stable CF variant.
Item–Item CF (Cosine, IDF-Weighted)	0.027495	Down-weighting popular items removed meaningful collaborative signal, leading to slightly weaker performance.
Truncated SVD (32 Components)	0.022373	Lower-dimensional embedding underfits interactions.
Truncated SVD (64 Components)	0.023711	Best SVD variant; good balance between variance captured and overfitting.
Truncated SVD (128 Components)	0.026600	Higher dimension introduces noise and does not improve ranking.

4. Discussion & Conclusion:

Decisions Made

- We selected three fundamentally different algorithmic families—LightGCN, ALS, and Collaborative Filtering—to understand how graph-based, latent-factor, and similarity-based methods behave on sparse implicit-feedback data.
- A unified preprocessing and evaluation pipeline (ID mapping, sparse matrices, consistent splitting, and NDCG@20 scoring) was used to ensure fairness across all models.
- Each team member independently implemented one model family to maximize diversity in design choices and implementation perspectives.

Difficulties Faced

- The dataset was extremely sparse, making it difficult for similarity-based methods to identify meaningful neighbors.
- The absence of negative feedback required careful handling of negative sampling, confidence weighting, and ranking-based losses.
- Hyperparameter tuning (factor size, regularization, propagation layers) significantly influenced performance and required extensive experimentation.
- Efficiently training models like LightGCN on large bipartite graphs required optimized sparse operations and memory management.

Things That Worked Well

- ALS performed the best overall, achieving the highest NDCG@20 among all models once factor size and regularization were tuned. Its confidence-weighted matrix factorization handled sparsity effectively and converged consistently.
- LightGCN showed strong performance, benefiting from multi-hop propagation on the user–item graph. Even with minimal neural components, it produced stable and competitive results.
- MF-SGD (within CF) was the strongest collaborative filtering variant, outperforming similarity-based approaches due to learned latent representations and negative sampling.
- The unified evaluation pipeline ensured consistent comparison and simplified debugging across all algorithms.

Things That Didn't Work Well

- Item–Item and Jaccard-based CF models performed the worst, largely because the dataset lacked overlapping user interactions, making similarity scores unreliable.
- Overly aggressive weighting sometimes removed valuable popularity signal, reducing NDCG@20.
- Increasing model depth or latent dimensions beyond a certain point often caused diminishing returns or noise rather than improvement.
- Some LightGCN–popularity hybrid variants underperformed due to overemphasis on globally popular items.

Conclusion

Across all experiments, ALS emerged as the strongest model, offering the best balance of ranking performance, stability, and computational efficiency for implicit-feedback recommendation. LightGCN performed closely behind ALS and demonstrated that graph-based propagation is highly effective for sparse datasets. Collaborative Filtering, while easy to interpret and simple to implement, struggled with the dataset's sparsity but served as an important baseline.

Overall, the project showed that latent-factor models (ALS and MF-SGD) and graph-based models (LightGCN) are better suited for large-scale implicit recommendation tasks than similarity-based CF methods. The comparative analysis provided valuable insight into how different algorithms handle sparse interactions, noisy signals, and large item catalogs—reflecting the real challenges faced in modern recommender systems.

5. Project Plan & Distribution:

Task	Assigned To	Planned Responsibility	Completed by
EDA	Assigned to all	Analyze interaction patterns, sparsity, user/item distributions.	All
Data Preprocessing	Assigned to all	Clean dataset, map IDs, prepare inputs for all models.	All
Collaborative Filtering	Harshitha	Implement user–user or item–item similarity-based recommendations; compute similarity matrices; generate top-20 predictions; evaluate using NDCG@20.	Harshitha
ALS Model	Bhargav	Implement ALS training pipeline, tune latent factors and regularization, run model variants, generate recommendation outputs, and evaluate results.	Bhargav
Light GCN	Hemanth	Construct user–item bipartite graph, implement LightGCN training (BPR loss & propagation layers), tune hyperparameters, and produce ranked recommendations.	Hemanth
Results & Comparison	Assigned to all	Compare NDCG@20 scores across all models, analyze which approaches worked best, summarize strengths/weaknesses, and interpret overall results.	All
Report Writing	Assigned to all	Write and format each model's section, compile methodology and results, edit for consistency, and prepare all final figures and tables.	All
PPT Presentation	Assigned to all	Prepare slides summarizing dataset, methods, results, and conclusions; divide speaking roles; practice and finalize presentation.	All

AI Tool Usage Declaration

Submission ID: 018223049-1764653936690

Date: 1/12/2025, 9:38:56 PM

Student Information

Student ID: 018223049

Name: Hemanth Sai Kosari

Course Code: CMPE-256

Assignment: Recommender System for
User-Item Interactions

Instructor: Gheorghe Guzun

Tool Name	AI Type	Purpose	Settings
ChatGPT (OpenAI)	Data Analysis	coding_help	-

I hereby confirm that the above AI tools were used ethically and in compliance with SJSU's academic integrity policy.

Digital Signature:

Hemanth Sai Kosari

Digitally signed on December 1, 2025 at 9:38 PM PST

AI Tool Usage Declaration

Submission ID: 018222087-1764657487036

Date: 12/2/2025, 12:08:07 PM

Student Information

Student ID: 018222087

Name: Sai Harshitha Peddi

Course Code: CMPE 256

Assignment: Group Project - Report

Instructor: Dr. Gheorghi Guzun

Tool Name	AI Type	Purpose	Settings
ChatGPT (OpenAI)	Text Generation	To improve grammar, clarity, or tone To explore multiple perspectives or ideas	-
ChatGPT (OpenAI)	Code Generation	To explore multiple perspectives or ideas	-

Additional Comments

Clarifying project requirements and structuring my workflow. wanted to try different methods we can work on.

I hereby confirm that the above AI tools were used ethically and in compliance with SJSU's academic integrity policy.

Digital Signature:

Sai Harshitha Peddi

Digitally signed on December 2, 2025 at 12:07 PM GMT+5:30

AI Tool Usage Declaration

Submission ID: 018176379-1764631368187

Date: 12/1/2025, 3:22:48 PM

Student Information

Student ID: 018176379

Name: Bhargava Phaneendra
Kumar Suryadevara

Course Code: CMPE 256

Assignment: Group Project - Report
submission

Instructor: Gheorghi Guzun

Tool Name	AI Type	Purpose	Settings
ChatGPT (OpenAI)	Text Generation	coding_help To improve grammar, clarity, or tone	-

I hereby confirm that the above AI tools were used ethically and in compliance with SJSU's academic integrity policy.

Digital Signature:

Bhargava Phaneendra Kumar Suryadevara

Digitally signed on December 1, 2025 at 12:48 PM PST